Ejercicio 1 (básico) – Identificar servicios y puertos críticos en Kali

Enunciado

En tu Kali:

1. Lista las conexiones y puertos en escucha.
2. Identifica qué servicios podrían ser **críticos** (por ejemplo, SSH).
3. Comprueba qué servicio escucha en el puerto 22.

Relaciona esto con la idea de “servicios críticos” del módulo de implantación IDS/IPS.

Solución (comandos)

1. Ver puertos en escucha y conexiones:

sudo ss -tulnp

```
~/Doc/T/transcendence/4-GDPR main !4 ?3 ❯ sudo ss -tulnp                                    44m 25s
[sudo] password for kali:
Netid  State   Recv-Q  Send-Q   Local Address:Port      Peer Address:Port  Process
udp    UNCONN  0       0              0.0.0.0:1194           0.0.0.0:*      users:(("openvpn",pid=767,fd=6))
tcp    LISTEN  0       4096           0.0.0.0:3443           0.0.0.0:*      users:(("docker-proxy",pid=229833
tcp    LISTEN  0       4096           0.0.0.0:6379           0.0.0.0:*      users:(("docker-proxy",pid=228762
tcp    LISTEN  0       200          127.0.0.1:5432           0.0.0.0:*      users:(("postgres",pid=837,fd=7))
tcp    LISTEN  0       4096           0.0.0.0:3000           0.0.0.0:*      users:(("docker-proxy",pid=229826
tcp    LISTEN  0       4096         127.0.0.1:39839          0.0.0.0:*      users:(("containerd",pid=744,fd=1
tcp    LISTEN  0       4096           0.0.0.0:443            0.0.0.0:*      users:(("docker-proxy",pid=229973
tcp    LISTEN  0       4096           0.0.0.0:80             0.0.0.0:*      users:(("docker-proxy",pid=229959
tcp    LISTEN  0       4096           0.0.0.0:8200           0.0.0.0:*      users:(("docker-proxy",pid=228719
tcp    LISTEN  0       80           127.0.0.1:3306           0.0.0.0:*      users:(("mariadbd",pid=178587,fd=
tcp    LISTEN  0       4096              [::]:3443              [::]:*      users:(("docker-proxy",pid=229842
tcp    LISTEN  0       200              [::1]:5432              [::]:*      users:(("postgres",pid=837,fd=6))
tcp    LISTEN  0       4096              [::]:6379              [::]:*      users:(("docker-proxy",pid=228772
tcp    LISTEN  0       4096              [::]:3000              [::]:*      users:(("docker-proxy",pid=22982
```

2. Ver qué servicio está asociado al puerto 22:

sudo ss -tulnp | grep ':80 '

```
~/Doc/T/transcendence/4-GDPR main !4 ?3 ❯ sudo ss -tulnp | grep ':80 '                          ✘ 0|1
tcp   LISTEN 0      4096         0.0.0.0:80         0.0.0.0:*    users:(("docker-proxy",pid=229959,fd=4))
tcp   LISTEN 0      4096            [::]:80            [::]:*    users:(("docker-proxy",pid=229966,fd=4))
```

3. Listar servicios activos en systemd:

systemctl --type=service –state=running

```
~/Doc/T/transcendence/4-GDPR main !4 ?3 ❯ systemctl --type=service --state=running
  UNIT                                  LOAD   ACTIVE SUB     DESCRIPTION
  accounts-daemon.service               loaded active running Accounts Service
  colord.service                        loaded active running Manage, Install and Generate Color Profiles
  containerd.service                    loaded active running containerd container runtime
  cron.service                          loaded active running Regular background program processing daemon
  dbus.service                          loaded active running D-Bus System Message Bus
  docker.service                        loaded active running Docker Application Container Engine
  getty@tty1.service                    loaded active running Getty on tty1
  haveged.service                       loaded active running Entropy Daemon based on the HAVEGE algorithm
  lightdm.service                       loaded active running Light Display Manager
  mariadb.service                       loaded active running MariaDB 11.8.3 database server
  me.proton.vpn.split_tunneling.service loaded active running Proton VPN Daemon
  ModemManager.service                  loaded active running Modem Manager
  NetworkManager.service                loaded active running Network Manager
  openvpn@server.service                loaded active running OpenVPN connection to server
  pcscd.service                         loaded active running PC/SC Smart Card Daemon
  polkit.service                        loaded active running Authorization Manager
  postgresql@17-main.service            loaded active running PostgreSQL Cluster 17-main
  rsyslog.service                       loaded active running System Logging Service
```

4. Si quieres ver específicamente ssh:

systemctl status ssh

```
~/Doc/T/transcendence/4-GDPR main !4 ?3 ❯ systemctl status ssh
○ ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled)
     Active: inactive (dead)
       Docs: man:sshd(8)
             man:sshd_config(5)
```

o en algunas Kali

systemctl status ssh.service

Explicación

- ss -tulnp te muestra:
  - t TCP, u UDP, l listening, n numérico, p proceso.
- A partir de ahí decides qué servicios son **críticos** (porque dan acceso remoto, manejan datos sensibles, etc.), como menciona el módulo 2 al hablar de priorizar servicios según criticidad y exposición a Internet.
- Este análisis previo es exactamente lo que se pide antes de desplegar IDS/IPS: saber **qué proteger primero**.

Ejercicio 2 (intermedio) – Mini IDS con Snort en modo escucha

Enunciado

En tu Kali, monta una prueba simple:

1. Ejecuta **Snort** en modo sniffer sobre tu interfaz (por ejemplo eth0 o ens33).
2. Desde otra máquina del laboratorio (o desde otra consola), haz un escaneo de puertos a Kali con nmap.
3. Comprueba que Snort detecta el escaneo y genera eventos.

Relaciona esto con los conceptos de IDS de red y detección de intrusiones.

Solución (comandos)

1. Instalar Snort si no está:

sudo apt update

sudo apt install snort -y

Durante la instalación te puede preguntar por la interfaz, puedes poner por ejemplo eth0 (o la tuya).

```
~/Documents/PROTON main ?1 ❯ sudo apt install snort -y
[sudo] password for kali:
Installing:
  snort

Installing dependencies:
  libdaq3  oinkmaster  snort-common  snort-common-libraries  snort-rules-default

Suggested packages:
  snort-doc

Summary:
  Upgrading: 0, Installing: 6, Removing: 0, Not Upgrading: 0
  Download size: 2,816 kB
  Space needed: 13.4 MB / 4,040 MB available
```

2. Ver qué interfaz usas:

ip a

3. Ejecutar Snort en modo sniffer/ver eventos en consola (sin reglas personalizadas, solo para ver tráfico):

sudo snort -i eth0 -A console

(sustituye eth0 por tu interfaz real)

```
~/Documents/PROTON main ?1 ❯ sudo snort -i eth0
--------------------------------------------------
o")~   Snort++ 3.1.82.0
--------------------------------------------------
--------------------------------------------------
pcap DAQ configured to passive.
Commencing packet processing
++ [0] eth0
```

```
~/Documents/PROTON main ?1 ❯ sudo snort -i eth0
--------------------------------------------------
o")~   Snort++ 3.1.82.0
--------------------------------------------------
--------------------------------------------------
pcap DAQ configured to passive.
Commencing packet processing
++ [0] eth0
^C** caught int signal
== stopping
-- [0] eth0
--------------------------------------------------
Packet Statistics
--------------------------------------------------
daq
                 received: 100
                 analyzed: 100
                    allow: 100
                 rx_bytes: 8106
--------------------------------------------------
codec
                    total: 100          (100.000%)
                    other: 1            (  1.000%)
                  discards: 7           (  7.000%)
                      arp: 53           ( 53.000%)
                      eth: 100          (100.000%)
                    icmp6: 18           ( 18.000%)
                     igmp: 2            (  2.000%)
                     ipv4: 28           ( 28.000%)
                     ipv6: 18           ( 18.000%)
                      tcp: 16           ( 16.000%)
                      udp: 10           ( 10.000%)
--------------------------------------------------
Module Statistics
--------------------------------------------------
detection
                 analyzed: 100
--------------------------------------------------
tcp
        bad_tcp4_checksum: 7
--------------------------------------------------
Summary Statistics
--------------------------------------------------
process
                  signals: 1
--------------------------------------------------
timing
                  runtime: 00:00:34
                  seconds: 34.446294
                 pkts/sec: 3
```

4. Desde **otra máquina** de tu laboratorio (o desde otra consola apuntando a la IP de Kali), lanza:

```
nmap -sS -p 1-1000 IP_DE_KALI
```

5. Observa la salida de Snort: debería mostrar paquetes de escaneo de puertos.

También puedes decirle que loguee en un directorio:

```
sudo snort -i eth0 -A console -l /var/log/snort
```

Explicación

- Aquí Kali está actuando como **NIDS** (IDS de red), analizando tráfico que pasa por una interfaz. Esto encaja con el módulo 1 cuando clasifica IDS por ubicación (perimetrales, internos, etc.).
- Un escaneo Nmap es un claro ejemplo de **intento de intrusión o reconocimiento**. El IDS detecta ese patrón y lo registra, igual que se explica en la parte de gestión de incidentes (detección → registro → análisis).

Ejercicio 3 (básico/intermedio) – Probar un antimalware en Kali con EICAR

(Relacionado con Módulo 3: control de código malicioso)

Enunciado

1. Instala y actualiza **ClamAV** en Kali.
2. Genera el archivo de prueba **EICAR** (no es malware real, es un test estándar).
3. Lanza un escaneo de ClamAV para detectar el archivo.
4. Observa cómo se registra el evento y qué acción toma.

Solución (comandos)

1. Instalar ClamAV:

sudo apt update

sudo apt install clamav clamav-daemon -y

```
~/Documents/IFCT0109 ❯ sudo apt install clamav clamav-daemon -y
[sudo] password for kali:
Installing:
  clamav  clamav-daemon

Installing dependencies:
  clamav-base  clamav-freshclam  clamdscan  libclamav12  libmspack0t64

Suggested packages:
  libclamunrar  clamav-doc  daemon  libclamunrar11

Summary:
  Upgrading: 0, Installing: 7, Removing: 0, Not Upgrading: 0
  Download size: 15.4 MB
  Space needed: 69.6 MB / 4,022 MB available
```

2. Parar el daemon mientras se actualiza la base (por si está corriendo):

sudo systemctl stop clamav-freshclam

3. Actualizar firmas:

sudo freshclam

```
~/Documents/IFCT0109 ❯ sudo systemctl stop clamav-freshclam
~/Documents/IFCT0109 ❯ sudo freshclam
Fri Nov 21 12:58:28 2025 -> ClamAV update process started at Fri Nov 21 12:58:28 2025
Fri Nov 21 12:58:28 2025 -> daily.cvd database is up-to-date (version: 27827, sigs: 2077134, f-leve
.clamav-publisher)
Fri Nov 21 12:58:28 2025 -> main.cvd database is up-to-date (version: 62, sigs: 6647427, f-level: 
Fri Nov 21 12:58:28 2025 -> bytecode.cvd database is up-to-date (version: 339, sigs: 80, f-level: 
p)
~/Documents/IFCT0109 ❯
```

4. Volver a arrancar el daemon (opcional):

sudo systemctl start clamav-freshclam

5. Crear un directorio de pruebas y generar el archivo EICAR:

mkdir ~/malware_test

cd ~/malware_test

nano eicar.com

Pega esta línea (es el test oficial EICAR – seguro y reconocido por antivirus):

X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*

Guarda y cierra.

```
~/Documents/box ❯ mkdir ~/malware_test
~/Documents/box ❯ cd ~/malware_test
~/malware_test ❯ nano eicar.com
~/malware_test ❯ cat eicar.com
X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
```

6. Escanear el directorio con ClamAV:

clamscan -r ~/malware_test

7. Verás algo tipo:

- Detectado: Eicar-Test-Signature
- Acción por defecto: reportar (si quieres que borre, puedes probar con --remove en un entorno de prueba):

```
~/malware_test ❯ clamscan -r ~/malware_test
Loading:    13s, ETA:   0s [=========================>]    8.71M/8.71M sigs
Compiling:   3s, ETA:   0s [=========================>]       41/41 tasks

/home/kali/malware_test/eicar.com: Eicar-Signature FOUND

----------- SCAN SUMMARY -----------
Known viruses: 8708831
Engine version: 1.4.3
Scanned directories: 1
Scanned files: 1
Infected files: 1
Data scanned: 0.00 MB
Data read: 0.00 MB (ratio 0.00:1)
Time: 17.382 sec (0 m 17 s)
Start Date: 2025:11:21 13:00:13
End Date:   2025:11:21 13:00:30
```

clamscan -r --remove ~/malware_test

```
~/malware_test ❯ clamscan -r --remove ~/malware_test
Loading:    13s, ETA:   0s [=========================>]    8.71M/8.71M sigs
Compiling:   3s, ETA:   0s [=========================>]       41/41 tasks

/home/kali/malware_test/eicar.com: Eicar-Signature FOUND
/home/kali/malware_test/eicar.com: Removed.

----------- SCAN SUMMARY -----------
Known viruses: 8708831
Engine version: 1.4.3
Scanned directories: 1
Scanned files: 1
Infected files: 1
Data scanned: 0.00 MB
Data read: 0.00 MB (ratio 0.00:1)
Time: 18.033 sec (0 m 18 s)
Start Date: 2025:11:21 13:01:22
End Date:   2025:11:21 13:01:40
```

Explicación

El módulo 3 habla de:

- **Detección por firmas** (justo lo que hace ClamAV con EICAR).
- Importancia de la **actualización de firmas y motor**.

En la práctica:

- freshclam aplica la idea de **actualización automática y frecuente**.
- clamscan -r ilustra análisis programado/manual de directorios.
- El hecho de detectar EICAR muestra cómo un antivirus identifica patrones concretos (firmas). Esto encaja con los criterios del módulo 3 para configurar correctamente una herramienta antimalware (protección activa, escaneos, actualización).

Ejercicio 4 (intermedio) – Detección de intento de intrusión SSH en logs

(Respuesta e intento de intrusión: módulos 1, 2, 4 y 5)

Enunciado

1. En tu Kali, simula varios intentos de login SSH fallidos **contra sí mismo**.
2. Analiza el fichero /var/log/auth.log para localizar esos intentos: IP, usuario, hora.
3. Diseña una "política de corte" simple: si una IP hace más de X intentos fallidos, la bloqueas temporalmente con iptables o ufw.
4. Aplica la política manualmente para la IP atacante.

Solución (comandos)

1. Asegúrate de que el servidor SSH está instalado y arrancado:

sudo apt install openssh-server -y

sudo systemctl enable --now ssh

2. Desde **otra máquina** del laboratorio (o desde la misma Kali usando localhost), fuerza algunos intentos fallidos:

ssh usuarioinventado@IP_DE_KALI

pon varias contraseñas incorrectas

Hazlo varias veces para generar varias líneas de log.

```
~/malware_test ❯ sudo apt install openssh-server -y
[sudo] password for kali:
openssh-server is already the newest version (1:10.2p1-2).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
~/malware_test ❯ sudo systemctl enable --now ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/syste
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
Created symlink '/etc/systemd/system/sshd.service' → '/usr/lib/systemd/system/s
Created symlink '/etc/systemd/system/multi-user.target.wants/ssh.service' → '/u
~/malware_test ❯ ssh kali@localhost
kali@localhost's password:
Last login: Wed Nov 19 14:49:27 CET 2025 from :0 on tty7
Linux kali 6.16.8+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.16.8-1kali1 (2025-09

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue Nov  4 12:25:11 2025 from ::1
```

3. En Kali, mira los intentos fallidos en /var/log/auth.log:

sudo grep "Failed password" /var/log/auth.log | tail -n 20

Verás líneas típicas con:

- Fecha/hora
- Servicio (sshd)
- Usuario
- IP origen

4. Cuenta cuántos intentos vienen de una misma IP (puedes filtrar):

sudo grep "Failed password" /var/log/auth.log | grep "IP_ATACANTE"

```
~/malware_test ❯ sudo grep "Failed password" /var/log/auth.log | tail -n 20                    ✘ 255 14s
2025-11-21T13:03:48.816099+01:00 kali sshd-session[350399]: Failed password for kali from ::1 port 46550 ssh2
2025-11-21T13:03:53.549129+01:00 kali sshd-session[350399]: Failed password for kali from ::1 port 46550 ssh2
2025-11-21T13:03:58.292823+01:00 kali sshd-session[350399]: Failed password for kali from ::1 port 46550 ssh2
2025-11-21T13:04:13.563713+01:00 kali sudo:     kali : TTY=pts/2 ; PWD=/home/kali/malware_test ; USER=root ;
usr/bin/grep 'Failed password' /var/log/auth.log
```

5. Política de corte manual: si la IP IP_ATACANTE tiene más de 5 fallos → bloqueamos con iptables:

Si usas ufw:

sudo ufw deny from IP_ATACANTE

(en un laboratorio, luego puedes quitar la regla con iptables -D o ufw delete deny ...)

Explicación

Aquí estás aplicando varias ideas de los módulos:

- El fichero de log es un **registro de auditoría** que te ayuda a detectar intentos de intrusión (módulos 2 y 3 hablan de la importancia de los logs para IDS/IPS y antimalware).
- Estás haciendo la fase de **detección e identificación** del incidente descrita en el ciclo de vida: ves un patrón sospechoso (muchos login fallidos).
- La regla en iptables es una implementación manual de una **política de corte**: umbral (nº de intentos), condición (misma IP), acción (bloquear). Igual que describe el módulo 2 para los IPS.

Ejercicio 5 (intermedio) – Mini análisis forense con dd + fls (Sleuthkit)

(Basado en módulo 6: análisis forense informático)

Enunciado

1. En tu Kali, crea un “disco” de prueba (un archivo) y dale formato ext4.
2. Monta ese disco, crea algunos ficheros y borra uno.
3. Usa dd para hacer una **imagen forense** del disco.
4. Con fls (Sleuthkit), lista el contenido de la imagen y localiza el archivo borrado.

Solución (comandos)

Todo esto es **en local**, con un archivo, nada peligroso.

1. Crear un archivo de 50 MB que hará de “disco”:

mkdir ~/forense_lab

cd ~/forense_lab

dd if=/dev/zero of=disco.raw bs=1M count=50

```
~/malware_test ❯ mkdir ~/forense_lab
~/malware_test ❯ cd ~/forense_lab
~/forense_lab ❯ dd if=/dev/zero of=disco.raw bs=1M count=50
50+0 records in
50+0 records out
52428800 bytes (52 MB, 50 MiB) copied, 0.0155242 s, 3.4 GB/s
```

2. Crear un sistema de ficheros ext4 dentro:

mkfs.ext4 disco.raw

```
~/forense_lab ❯ mkfs.ext4 disco.raw
mke2fs 1.47.2 (1-Jan-2025)
Discarding device blocks: done
Creating filesystem with 51200 1k blocks and 12824 inodes
Filesystem UUID: f2e15c77-f33c-4f8d-88ad-c226c4a2c696
Superblock backups stored on blocks:
        8193, 24577, 40961

Allocating group tables: done
Writing inode tables: done
Creating journal (4096 blocks): done
Writing superblocks and filesystem accounting information: done
```

3. Crear un punto de montaje y montar:

sudo mkdir /mnt/forense_lab

sudo mount -o loop disco.raw /mnt/forense_lab

4. Crear algunos archivos:

sudo touch /mnt/forense_lab/archivo1.txt

sudo touch /mnt/forense_lab/archivo2.txt

echo "hola forense" | sudo tee /mnt/forense_lab/archivo_secreto.txt

```
~/forense_lab ❯ sudo mkdir /mnt/forense_lab
sudo mount -o loop disco.raw /mnt/forense_lab
~/forense_lab ❯ sudo touch /mnt/forense_lab/archivo1.txt
sudo touch /mnt/forense_lab/archivo2.txt
echo "hola forense" | sudo tee /mnt/forense_lab/archivo_secreto.txt
hola forense
```

5. Borrar uno de ellos (por ejemplo archivo2.txt):

sudo rm /mnt/forense_lab/archivo2.txt

6. Desmontar el “disco”:

sudo umount /mnt/forense_lab

7. Hacer una **imagen** con dd (simulando adquisición forense):

dd if=disco.raw of=imagen.dd bs=1M

(en este caso la imagen es una copia 1:1 del mismo archivo, pero vale para el ejercicio)

```
~/forense_lab ❯ sudo rm /mnt/forense_lab/archivo2.txt
~/forense_lab ❯ sudo umount /mnt/forense_lab
~/forense_lab ❯ dd if=disco.raw of=imagen.dd bs=1M
50+0 records in
50+0 records out
52428800 bytes (52 MB, 50 MiB) copied, 0.0262812 s, 2.0 GB/s
```

8. Usar fls (Sleuthkit) para listar ficheros, incluyendo borrados:

fls -r -m / imagen.dd

- -r recursivo
- -m / añade un “prefijo” de ruta

Verás entradas tipo:

- r/r = archivo existente
- d/d = directorio

- d/d * o r/r * (con *) suelen ser borrados (depende de versión).

•

```
~/forense_lab ❯ fls -r -m / imagen.dd
0|/lost+found|11|d/drwx------|0|0|12288|1763726797|1763726797|1763726797|1763726797
0|/archivo1.txt|13|r/rrw-r--r--|0|0|0|1763726855|1763726855|1763726855|1763726855
0|/archivo_secreto.txt|15|r/rrw-r--r--|0|0|13|1763726855|1763726855|1763726855|1763726855
0|/$OrphanFiles|12825|V/V---------|0|0|0|0|0|0|0
0|/$OrphanFiles/OrphanFile-14 (deleted)|14|-/rrw-r--r--|0|0|0|1763726855|1763726855|1763726879|1763726855
```

Puedes filtrar:

fls -r imagen.dd | grep archivo

```
~/forense_lab ❯ fls -r imagen.dd | grep archivo
r/r 13: archivo1.txt
r/r 15: archivo_secreto.txt
```

Explicación

Estás imitando a pequeña escala las fases del módulo 6:

- **Adquisición de evidencia**: dd if=disco.raw of=imagen.dd copia bit a bit el “disco”.
- **Análisis de evidencias**: fls examina la estructura del sistema de archivos en la imagen, permitiendo ver archivos borrados.
- En un caso real, esto se haría sobre un disco, USB o imagen de un sistema comprometido, respetando principios de cadena de custodia, no alteración de evidencia, etc., justo lo que se explica en el módulo de análisis forense.